Database Query Analysis and Design Document

Overview

This document provides a detailed analysis of 10 critical database queries designed for the hotel management system. These queries address core business requirements including room usage analysis, reservation patterns, billing management, and maintenance operations. Each query demonstrates the integration of multi-table joins, aggregation statistics, and business logic.

Query 1: Room Usage by Building and Wing

Query Purpose

Summarizes the frequency of room assignments across different buildings and wings in the hotel complex, helping management understand the distribution of space utilization.

SQL Statement

```sql
SELECT
    b.buildingName AS building_name,
    w.wingCode AS wing_code,
    COUNT(DISTINCT ra.assignmentId) AS total_assignments
FROM room_assignment AS ra
JOIN room AS r
    ON ra.roomId = r.roomId
JOIN floor AS f
    ON r.floorId = f.floorId
JOIN wing AS w
    ON f.wingId = w.wingId
JOIN building AS b
    ON w.buildingId = b.buildingId
GROUP BY
    b.buildingName,
    w.wingCode
ORDER BY
    total_assignments DESC;
```

Key Design Elements

Hierarchical Multi-Table Join: Through 4 levels of JOIN operations, the query traces room assignments back through the organizational hierarchy: Hotel → Building → Wing → Floor → Room. This reflects the real-world structure of the hotel facility.

DISTINCT Counting: Using `COUNT(DISTINCT ra.assignmentId)` instead of simple `COUNT(*)` ensures accurate statistics without inflation from potential data duplication.

Business Application: Identifies high-usage areas to guide resource allocation and maintenance scheduling.

Query 2: Average Actual Usage Duration by Party Type

Query Purpose

Calculates the average length of stay by party type (individuals, corporations, groups, etc.), revealing usage patterns across different customer segments.

SQL Statement

```
SELECT
    p.partyType AS party_type,
    ROUND(AVG(julianday(s.checkoutTime) - julianday(s.checkinTime)), 2) AS
avg_usage_days,
    COUNT(DISTINCT s.stayId) AS total_stays
FROM stay AS s
JOIN reservation AS res
    ON s.reservationId = res.reservationId
JOIN party AS p
    ON res.partyId = p.partyId
GROUP BY
    p.partyType
ORDER BY
    avg_usage_days DESC;
```

Key Design Elements

Date/Time Arithmetic: SQLite's `julianday()` function converts dates to Julian day numbers, allowing calculation of the difference in days between check-in and check-out times. This approach is standardized across database systems.

Data Precision: Using `ROUND(..., 2)` maintains two decimal places for readability while avoiding irregular floating-point displays.

Aggregation Robustness: `COUNT(DISTINCT s.stayId)` ensures accurate counting in complex relationships where multiple reservations may correspond to multiple stays.

Operational Insight: Identifies average stay duration for different customer segments, supporting pricing strategies and inventory planning.

Query 3: Reservation Lead Time and Deposit Pattern

Query Purpose

Groups reservations by booking lead time (days in advance) and analyzes the frequency and average amount of deposits required in each group, revealing relationships between lead time and deposit policies.

SQL Statement

```
SELECT
    CASE
        WHEN julianday(startDate) - julianday(dateCreated) < 7 THEN 'Under
7 days'
        WHEN julianday(startDate) - julianday(dateCreated) < 30 THEN '7-29
days'
        WHEN julianday(startDate) - julianday(dateCreated) < 90 THEN '30-
89 days'
        ELSE '90+ days'
    END AS lead_time_group,
    COUNT(*) AS reservation_count,
    SUM(CASE WHEN depositRequired = 1 THEN 1 ELSE 0 END) AS
reservations_with_deposit,
    ROUND(AVG(depositAmount), 2) AS avg_deposit_amount
FROM reservation
GROUP BY
    lead_time_group
ORDER BY
    reservation_count DESC;
```

Key Design Elements

CASE Expression Strategy: Transforms continuous time ranges into discrete categorical intervals. This approach creates both a grouping dimension and a meaningful, interpretable categorization.

Conditional Aggregation: `SUM(CASE WHEN depositRequired = 1 THEN 1 ELSE 0 END)` implements record counting based on conditions, a pattern that is more universal than IF-based approaches.

Comprehensive Metrics: Simultaneously tracks reservation count, deposit requirement frequency, and average deposit amount, providing a multidimensional business perspective.

Policy Optimization: Helps adjust deposit policies by revealing patterns, such as discovering whether longer-lead reservations have lower deposit requirements.

Query 4: Most Common Bed Configurations in Assigned Rooms

Query Purpose

Identifies which bed types appear most frequently in rooms actually assigned to guests, reflecting real-world room configuration requirements.

SQL Statement

```
SELECT
    bt.name AS bed_type,
```

```
    SUM(rhb.quantity) AS total_beds_in_assigned_rooms,
    COUNT(DISTINCT r.roomId) AS rooms_with_this_bed_type
FROM room_assignment AS ra
JOIN room AS r
    ON ra.roomId = r.roomId
JOIN room_has_bed AS rhb
    ON r.roomId = rhb.roomId
JOIN bed_type AS bt
    ON rhb.bedTypeId = bt.bedTypeId
GROUP BY
    bt.name
ORDER BY
    total_beds_in_assigned_rooms DESC,
    rooms_with_this_bed_type DESC;
```

Key Design Elements

Filtering to Actually-Used Rooms: By filtering through the `room_assignment` table, the query counts only rooms actually assigned to guests, excluding unused or maintenance rooms. This ensures statistical accuracy and relevance.

Multiple Aggregation Dimensions: Provides both total bed counts in assigned rooms (`SUM(rhb.quantity)`) and the count of rooms with this bed type (`COUNT(DISTINCT r.roomId)`), enabling flexible analysis from different angles.

Multi-Level Sorting: Orders first by total bed count descending, then by room count descending, ensuring the most important configurations appear at the top.

Facility Optimization: Guides procurement decisions and room renovation planning.

Query 5: Event Size and Room Usage by Event Type

Query Purpose

Analyzes different types of events (conferences, weddings, banquets, etc.) to understand typical room requirements and guest capacity for each event category.

SQL Statement

```
SELECT
    e.eventType AS event_type,
    COUNT(DISTINCT e.eventId) AS total_events,
    ROUND(AVG(e.estimatedGuestCount), 2) AS avg_estimated_guests,
    ROUND(AVG(rc.room_count), 2) AS avg_rooms_used
FROM event AS e
JOIN (
    SELECT
        eventId AS event_id,
        COUNT(roomId) AS room_count
```

```
    FROM event_room
    GROUP BY
        eventId
) AS rc
    ON e.eventId = rc.event_id
GROUP BY
    e.eventType
ORDER BY
    avg_rooms_used DESC,
    avg_estimated_guests DESC;
```

Key Design Elements

Subquery Application: The subquery `(SELECT eventId, COUNT(roomId) AS room_count FROM event_room GROUP BY eventId)` pre-calculates the room count for each event. This improves query readability by avoiding complex calculations in the outer GROUP BY.

Subquery Aliasing: Naming the subquery as `rc` makes JOIN conditions clear and explicit.

Multi-Criteria Sorting: Considers both room usage and guest count to determine event "importance"—providing nuanced prioritization.

Capacity Planning: Supports allocation of appropriate space and facilities for different event types.

Query 6: Revenue by Service Type

Query Purpose

Summarizes total charges, revenue, and average charge amount for each service category (rooms, dining, parking, etc.), identifying which services generate the highest revenue.

SQL Statement

```
SELECT
    st.serviceType AS service_type,
    COUNT(c.chargeId) AS total_charges,
    SUM(c.chargeAmount) AS total_revenue,
    ROUND(AVG(c.chargeAmount), 2) AS avg_charge
FROM charge AS c
JOIN service_type AS st
    ON c.serviceCode = st.serviceCode
GROUP BY
    st.serviceType
ORDER BY
    total_revenue DESC;
```

Key Design Elements

Focused Join: Though using only one JOIN, connecting charge records to service type master data via service code provides complete visibility into the revenue structure.

Three-Dimensional Revenue Perspective: Count, total amount, and average charge provide different angles on revenue characteristics. For example, high average charge with low count may indicate premium services, while high count with low average indicates foundational services.

Descending Sort: Default ordering by total revenue descending provides management with an immediate overview of revenue contribution rankings.

Revenue Management: Directly supports revenue analysis, pricing review, and service optimization decisions.

Query 7: Top 10 Highest-Value Billing Parties

Query Purpose

Identifies the top 10 customers (individuals or organizations) by total billing amount, revealing customer value rankings and enabling targeted relationship management.

SQL Statement

```sql
SELECT
    p.partyId AS party_id,
    p.partyType AS party_type,
    p.contactPersonName AS contact_person_name,
    p.email AS email,
    COUNT(DISTINCT ba.accountId) AS total_accounts,
    COUNT(c.chargeId) AS total_charges,
    SUM(c.chargeAmount) AS total_billed_amount
FROM party AS p
JOIN billing_account AS ba
    ON p.partyId = ba.partyId
JOIN charge AS c
    ON ba.accountId = c.accountId
GROUP BY
    p.partyId,
    p.partyType,
    p.contactPersonName,
    p.email
ORDER BY
    total_billed_amount DESC
LIMIT 10;
```

Key Design Elements

Multiple Count Dimensions:

- `COUNT(DISTINCT ba.accountId)`: One customer may have multiple accounts

- `COUNT(c.chargeId)`: Each account may have multiple charge records

These separate counts reflect the true business model where the customer → account → charge relationship is many-to-many.

Complete Customer Information: Beyond financial metrics, includes contact person name and email, supporting follow-up communications or relationship management.

LIMIT Usage: Restricting results to the top 10 highlights key customers while maintaining query performance.

Customer Relationship Management: Identifies high-value customers, supports VIP services, and guides marketing strategy.

Query 8: Maintenance Burden by Building and Wing

Query Purpose

Summarizes the number of maintenance tickets and average resolution time for each building and wing, identifying areas with the highest maintenance workload and efficiency issues.

SQL Statement

```sql
SELECT
    b.buildingName AS building_name,
    w.wingCode AS wing_code,
    COUNT(mt.ticketId) AS total_tickets,
    ROUND(AVG(julianday(mt.dateResolved) - julianday(mt.dateCreated)), 2)
AS avg_resolution_days
FROM maintenance_ticket AS mt
JOIN room AS r
    ON mt.roomId = r.roomId
JOIN floor AS f
    ON r.floorId = f.floorId
JOIN wing AS w
    ON f.wingId = w.wingId
JOIN building AS b
    ON w.buildingId = b.buildingId
GROUP BY
    b.buildingName,
    w.wingCode
ORDER BY
    total_tickets DESC,
    avg_resolution_days DESC;
```

Key Design Elements

Tracing From Ticket to Location: Through 4 levels of JOIN, the query traces from maintenance tickets backward to the wing and building where the rooms are located, establishing clear linkage between

maintenance work and physical space.

Time-to-Resolution Metric: `julianday(dateResolved) - julianday(dateCreated)` calculates the days from ticket creation to resolution—a key efficiency indicator.

Composite Sorting Priority: First orders by ticket count descending (identifying problem areas), then by resolution time descending (identifying efficiency gaps). This dual ordering provides refined prioritization.

Operations Management: Supports resource allocation, maintenance scheduling optimization, and problem identification.

Query 9: Top 10 Rooms with Highest Turnover

Query Purpose

Identifies the 10 most frequently assigned rooms, which typically require more frequent cleaning, faster turnaround preparation, and increased maintenance attention.

SQL Statement

```sql
SELECT
    r.roomId AS room_id,
    r.roomNumber AS room_number,
    COUNT(ra.assignmentId) AS total_assignments
FROM room AS r
JOIN room_assignment AS ra
    ON r.roomId = ra.roomId
GROUP BY
    r.roomId,
    r.roomNumber
ORDER BY
    total_assignments DESC
LIMIT 10;
```

Key Design Elements

Simple yet Effective Design: Although using only two tables with a basic JOIN, the query effectively summarizes assignment frequency per room—demonstrating economy in query design.

Comprehensive GROUP BY: While `roomId` alone is sufficient as a primary key, including `roomNumber` in the GROUP BY improves result readability and follows best practices.

LIMIT with Sorting: The combination of ORDER BY and LIMIT identifies the TOP 10 high-frequency rooms with clean, efficient syntax.

Resource Management: Guides cleaning schedules, maintenance prioritization, and asset upgrade strategies.

Query 10: Monthly Revenue Trend by Service Type

Query Purpose

Displays revenue trends over time for different service categories, helping identify seasonal patterns and track growth or decline trajectories for each service.

SQL Statement

```sql
SELECT
    strftime('%Y-%m', c.dateIncurred) AS revenue_year_month,
    st.serviceType AS service_type,
    COUNT(c.chargeId) AS total_charges,
    SUM(c.chargeAmount) AS total_revenue
FROM charge AS c
JOIN service_type AS st
    ON c.serviceCode = st.serviceCode
GROUP BY
    strftime('%Y-%m', c.dateIncurred),
    st.serviceType
ORDER BY
    revenue_year_month,
    total_revenue DESC;
```

Key Design Elements

Time Dimension Standardization: `strftime('%Y-%m', c.dateIncurred)` formats specific date/time values into year-month strings (e.g., '2024-03'). This is the standard pattern for time-based aggregation in SQLite.

Two-Dimensional Grouping: Grouping simultaneously by year-month and service type maintains temporal continuity while isolating revenue contributions by service category.

Sorting Strategy: Orders first by month ascending (preserving natural temporal sequence), then by revenue descending within each month (highlighting the primary revenue driver for that period).

Trend Analysis: Supports seasonality analysis, service growth tracking, anomaly detection, and budget planning.

Summary

These 10 queries comprehensively address the core business domains of the hotel management system:

Query	Primary Function	Key Techniques
1	Space Utilization Analysis	Multi-level JOIN, DISTINCT counting
2	Customer Behavior Analysis	Date/Time arithmetic, multi-table joins
3	Deposit Policy Analysis	CASE expressions, conditional aggregation

Query	Primary Function	Key Techniques
4	Facility Configuration Analysis	Multiple aggregation dimensions, DISTINCT counting
5	Event Planning Analysis	Subqueries, multi-criteria sorting
6	Service Revenue Analysis	Simple JOIN, aggregate statistics
7	Customer Value Analysis	Multi-dimensional aggregation, LIMIT
8	Maintenance Efficiency Analysis	Multi-level JOIN, time calculation
9	Room Hotspot Analysis	Economical design, LIMIT sorting
10	Revenue Trend Analysis	Time formatting, two-dimensional grouping

These queries collectively demonstrate how SQL's multi-table joins, aggregate functions, time handling, and grouping techniques extract meaningful business insights from complex relational data structures.